

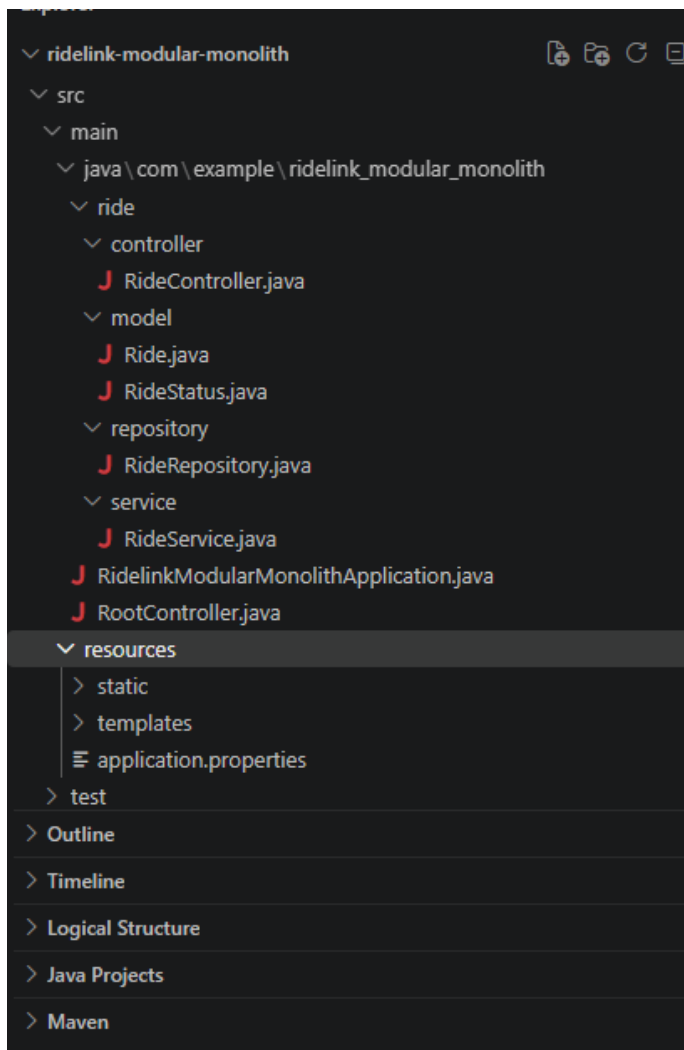


RideLink Lite: Building a Modular Monolith

IT3130 Application Development • Guided practical • No frontend required

Explorer

- ▼ ridelink-modular-monolith
 - > .github
 - > .mvn
 - ▼ .vscode
 - { } launch.json
 - { } settings.json
 - ▼ src
 - ▼ main
 - ▼ java\com\example\ridelink_modular_monolith
 - ▼ driver
 - ▼ controller
 - DriverController.java
 - ▼ model
 - Driver.java
 - ▼ repository
 - DriverRepository.java
 - ▼ service
 - DriverService.java



```
J RideController.java
src > main > java > com > example > ridelink_modular_monolith > ride > controller > J RideController.java > Language Support f
1  package com.example.ridelink_modular_monolith.ride.controller;
2
3  import org.springframework.web.bind.annotation.GetMapping;
4  import org.springframework.web.bind.annotation.PatchMapping;
5  import org.springframework.web.bind.annotation.PathVariable;
6  import org.springframework.web.bind.annotation.PostMapping;
7  import org.springframework.web.bind.annotation.RequestBody;
8  import org.springframework.web.bind.annotation.RequestMapping;
9  import org.springframework.web.bind.annotation.RestController;
10
11  import com.example.ridelink_modular_monolith.ride.model.Ride;
12  import com.example.ridelink_modular_monolith.ride.service.RideService;
13
14  @RestController
15  @RequestMapping("/api/rides")
16  public class RideController {
17
18      private final RideService service;
19
20      public RideController(RideService service) {
21          this.service = service;
22      }
23
24      @PostMapping
25      public Ride requestRide(
26          @RequestBody Ride ride) {
27
28          return service.requestRide(ride);
29      }
30
31      @GetMapping("/{id}")
```

```

    @GetMapping("/{id}")
    public Ride getRide(
        @PathVariable Long id) {
        return service.getRide(id);
    }

    @PatchMapping("/{id}/complete")
    public Ride completeRide(
        @PathVariable Long id) {
        return service.completeRide(id);
    }
}
```

```
J Ride.java
src > main > java > com > example > ridelink_modular_monolith > ride > model > J Ride.java > Ride

3  import jakarta.persistence.Id;
4  import jakarta.persistence.Entity;
5  import jakarta.persistence.EnumType;
6  import jakarta.persistence.Enumerated;
7  import jakarta.persistence.GeneratedValue;
8  import jakarta.persistence.GenerationType;
9
10 @Entity
11 public class Ride {
12
13     @Id
14     @GeneratedValue(strategy = GenerationType.IDENTITY)
15     private Long id;
16
17     private String passengerName;
18
19     private String pickup;
20
21     private String destination;
22
23     private Long driverId;
24
25     @Enumerated(EnumType.STRING)
26     private RideStatus status;
27
28     public Long getId() {
29         return id;
30     }
}
```

```
src > main > java > com > example > ridelink_modular_monolith > ride > model > J Ride.java >
11  public class Ride {
36      public String getPickup() {
38          }
39
40      public String getDestination() {
41          return destination;
42      }
43
44      public Long getDriverId() {
45          return driverId;
46      }
47
48      public RideStatus getStatus() {
49          return status;
50      }
51
52      public void setId(Long id) {
53          this.id = id;
54      }
55
56      public void setPassengerName(String passengerName) {
57          this.passengerName = passengerName;
58      }
59
60      public void setPickup(String pickup) {
61          this.pickup = pickup;
62      }
63
64      public void setDestination(String destination) {
65          this.destination = destination;
66      }
}
```

J RideStatus.java

```
src > main > java > com > example > ridelink_modu
1  package com.example.ridelink_modu
2
3  public enum RideStatus {
4      REQUESTED,
5      ASSIGNED,
6      COMPLETED
7  }
8
```

J RideRepository.java

```
src > main > java > com > example > ridelink_modular_monolith > ride > repository > J Ride
1  package com.example.ridelink_modular_monolith.ride.repository;
2
3  import com.example.ridelink_modular_monolith.ride.model.Ride;
4  import org.springframework.data.jpa.repository.JpaRepository;
5
6  public interface RideRepository
7      extends JpaRepository<Ride, Long> {
8  }
```

J RideService.java

```
src > main > java > com > example > ridelink_modular_monolith > ride > service > J RideService.java > Language
1  package com.example.ridelink_modular_monolith.ride.service;
2
3  import org.springframework.http.HttpStatus;
4  import org.springframework.stereotype.Service;
5  import org.springframework.web.server.ResponseStatusException;
6
7  import com.example.ridelink_modular_monolith.driver.model.Driver;
8  import com.example.ridelink_modular_monolith.driver.service.DriverService;
9  import com.example.ridelink_modular_monolith.ride.model.Ride;
10 import com.example.ridelink_modular_monolith.ride.model.RideStatus;
11 import com.example.ridelink_modular_monolith.ride.repository.RideRepository;
12
13 import java.util.List;
14
15 @Service
16 public class RideService {
17
18     private final RideRepository rideRepository;
19     private final DriverService driverService;
20
21     public RideService(
22         RideRepository rideRepository,
23         DriverService driverService) {
24
25         this.rideRepository = rideRepository;
26         this.driverService = driverService;
27     }
28
29     public Ride requestRide(Ride ride) {
30
31         List<Driver> availableDrivers = driverService.getAvailableDrivers();
```

```

main > java > com > example > nidelink_modular_monolith > ride > service > RideService.java > LanguageTool
public class RideService {
    public Ride requestRide(Ride ride) {

        List<Driver> availableDrivers = driverService.getAvailableDrivers();

        if (availableDrivers.isEmpty()) {
            throw new ResponseStatusException(
                HttpStatus.CONFLICT,
                "No driver available");
        }

        Driver driver = availableDrivers.get(index: 0);

        driver.setAvailable(available: false);
        driverService.save(driver);

        ride.setDriverId(driver.getId());
        ride.setStatus(RideStatus.ASSIGNED);

        return rideRepository.save(ride);
    }

    public Ride getRide(Long id) {
        return rideRepository.findById(id)
            .orElseThrow();
    }
}

```

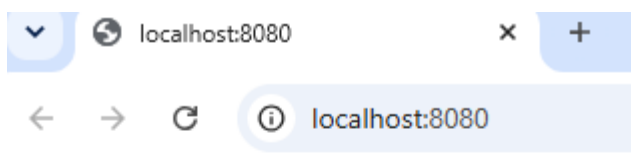
```

16 public class RideService {
29     public Ride requestRide(Ride ride) {
42         driver.setAvailable(available: false);
43         driverService.save(driver);
44
45         ride.setDriverId(driver.getId());
46         ride.setStatus(RideStatus.ASSIGNED);
47
48         return rideRepository.save(ride);
49     }
50
51     public Ride getRide(Long id) {
52         return rideRepository.findById(id)
53             .orElseThrow();
54     }
55
56     public Ride completeRide(Long id) {
57
58         Ride ride = getRide(id);
59
60         Driver driver = driverService.findById(ride.getDriverId());
61
62         driver.setAvailable(available: true);
63         driverService.save(driver);
64
65         ride.setStatus(RideStatus.COMPLETED);
66
67         return rideRepository.save(ride);
68     }
69 }

```

```
J RidelinkModularMonolithApplication.java
src > main > java > com > example > ridelink_modular_monolith > J RidelinkModularMonolithApplication.java
1 package com.example.ridelink_modular_monolith;
2
3 import org.springframework.boot.SpringApplication;
4 import org.springframework.boot.autoconfigure.SpringBootApplication;
5
6 @SpringBootApplication
7 public class RidelinkModularMonolithApplication {
8
9     Run | Debug
10     public static void main(String[] args) {
11         SpringApplication.run(RidelinkModularMonolithApplication.class, args);
12     }
13 }
```

```
J RootController.java
src > main > java > com > example > ridelink_modular_monolith > J RootController.java > Language Su
1 package com.example.ridelink_modular_monolith;
2
3 import org.springframework.web.bind.annotation.GetMapping;
4 import org.springframework.web.bind.annotation.RestController;
5
6 @RestController
7 public class RootController {
8
9     @GetMapping("/")
10     public String home() {
11         return "RideLink application is running";
12     }
13 }
14
```



RideLink application is running

GET Untitled Request POST Post data +

My Collection > Post data

POST http://localhost:8080/api/drivers

Docs Params Authorization Headers 9 Body Scripts Tests Se

Query Params

Key	Value
Key	Value

Body Cookies Headers 5 Test Results 1/1

{ } JSON Preview Visualize

```
1 {
2   "name": "Add your name in the body",
3   "vehicleNumber": null,
4   "available": true,
5   "id": 1
6 }
```

GET Untitled Request POST Post data +

My Collection > Post data

POST http://localhost:8080/api/drivers

Docs Params Authorization Headers 9 Body Scripts Tests Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON

```
1 {
2   "name": "Nimal",
3   "vehicleNumber": "CAB-1024"
4 }
```

Body Cookies Headers 5 Test Results 1/1

{ } JSON Preview Visualize

```
1 {
2   "name": "Add your name in the body",
3   "vehicleNumber": null,
4   "available": true,
5   "id": 1
6 }
```

GET Untitled Request POST Post data +

My Collection > Post data

POST http://localhost:8080/api/drivers

Docs Params Authorization Headers 9 Body Scripts Tests

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary

```
1 {
2   "name": "Nimal",
3   "vehicleNumber": "CAB-1024"
4 }
```

Body Cookies Headers 5 Test Results 1/1

{ } JSON Preview Visualize

```
1 {
2   "name": "Nimal",
3   "vehicleNumber": "CAB-1024",
4   "available": true,
5   "id": 2
6 }
```

My Collection > Post data

GET http://localhost:8080/api/drivers/available

Docs Params Authorization Headers 9 Body Scripts Tests S

Body Cookies Headers 5 Test Results 1/1

{ } JSON Preview Visualize

```
1 [
2   {
3     "name": "Add your name in the body",
4     "vehicleNumber": null,
5     "available": true,
6     "id": 1
7   },
8   {
9     "name": "Nimal",
10    "vehicleNumber": "CAB-1024",
11    "available": true,
12    "id": 2
13  }
14 ]
```

GET Untitled Request POST Post data + No environment

My Collection > Post data Save Share

POST http://localhost:8080/api/rides Send

Body

Body 200 OK • 116 ms • 251 B • e.g.

{ } JSON

```
1 {
2   "destination": null,
3   "driverId": 1,
4   "id": 1,
5   "passengerName": null,
6   "pickup": null,
7   "status": "ASSIGNED"
8 }
```

http://localhost:8080/api/rides/1

GET http://localhost:8080/api/rides/1

Docs Params Authorization Headers 7 Body Scripts Settings

Query Params

Key	Value
Key	Value

Body Cookies Headers 5 Test Results

{ } JSON Preview Visualize

```
1 {
2   "destination": null,
3   "driverId": 1,
4   "id": 1,
5   "passengerName": null,
6   "pickup": null,
7   "status": "ASSIGNED"
8 }
```

HTTP **http://localhost:8080/api/rides/2**

GET **http://localhost:8080/api/rides/2**

Docs Params Authorization Headers 7 Body Scripts Settings

Query Params

Key	Value
Key	Value

Body Cookies Headers 5 Test Results

{ } JSON Preview Visualize

```
1 {
2   "destination": "Malabe Junction",
3   "driverId": 2,
4   "id": 2,
5   "passengerName": "Ayesha",
6   "pickup": "SLIIT",
7   "status": "ASSIGNED"
8 }
```

HTTP **http://localhost:8080/api/rides/2/complete**

PATCH **http://localhost:8080/api/rides/2/complete**

Docs Params Authorization Headers 8 Body Scripts Settings

Query Params

Key	Value
Key	Value

Body Cookies Headers 5 Test Results

{ } JSON Preview Visualize

```
1 {
2   "destination": "Malabe Junction",
3   "driverId": 2,
4   "id": 2,
5   "passengerName": "Ayesha",
6   "pickup": "SLIIT",
7   "status": "COMPLETED"
8 }
```

HTTP **http://localhost:8080/api/drivers**

POST **http://localhost:8080/api/drivers**

Docs Params Authorization Headers 9 **Body** Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary

```
1 {
2   "name": "Kamal",
3   "vehicleNumber": "CAB-2048"
4 }
```

Body Cookies Headers 5 Test Results

{ } JSON Preview Visualize

```
1 {
2   "name": "Kamal",
3   "vehicleNumber": "CAB-2048",
4   "available": true,
5   "id": 3
6 }
```

HTTP **http://localhost:8080/api/drivers/available**

GET **http://localhost:8080/api/drivers/available**

Docs Params Authorization Headers 9 **Body** Scripts Settings

Body Cookies Headers 5 Test Results

{ } JSON Preview Visualize

```
3   "name": "Add your name in the body",
4   "vehicleNumber": null,
5   "available": true,
6   "id": 1
7 },
8 {
9   "name": "Nimal",
10  "vehicleNumber": "CAB-1024",
11  "available": true,
12  "id": 2
13 },
14 {
15   "name": "Kamal",
16   "vehicleNumber": "CAB-2048",
17   "available": true,
18   "id": 3
19 }
20 ]
```

POST http://localhost:8080/ap +

My Collection > http://localhost:8080/api/rides

POST http://localhost:8080/api/rides

Docs Params Authorization Headers 9 Body S

Body Cookies Headers 5 Test Results

{ } JSON Preview Visualize

```
1 {
2   "destination": "Nugegoda Junction",
3   "driverId": 2,
4   "id": 3,
5   "passengerName": "Chethi",
6   "pickup": "Maharagama Junction",
7   "status": "ASSIGNED"
8 }
```

PATCH http://localhost:8080C +

My Collection > http://localhost:8080/api/rides

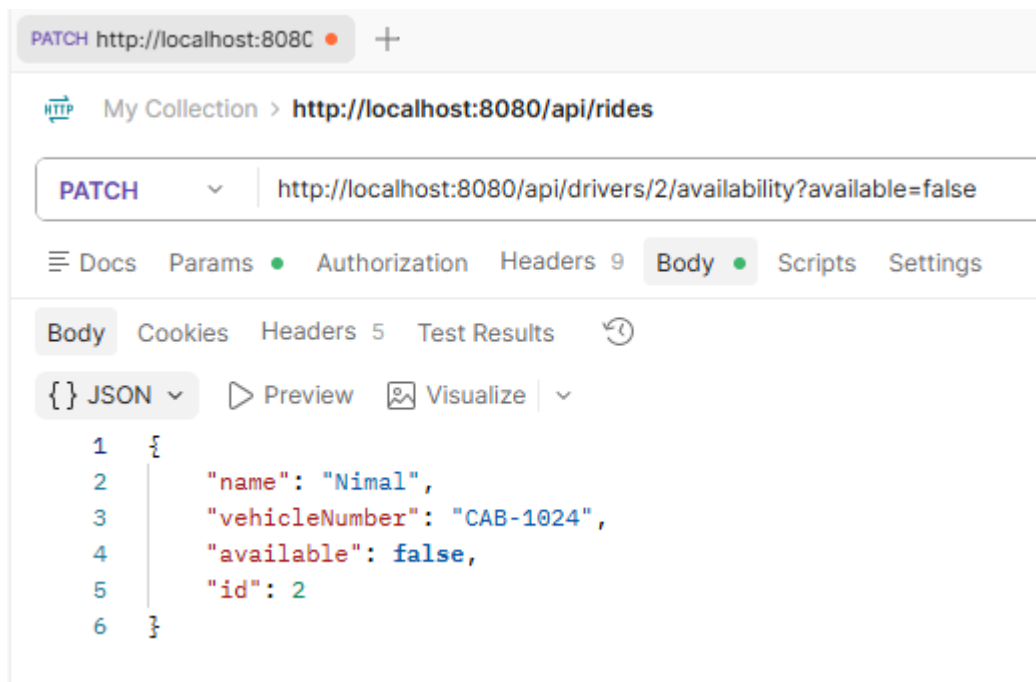
PATCH http://localhost:8080/api/drivers/1/availability?available=false

Docs Params Authorization Headers 9 Body Scripts Settings

Body Cookies Headers 5 Test Results

{ } JSON Preview Visualize

```
1 {
2   "name": "Add your name in the body",
3   "vehicleNumber": null,
4   "available": false,
5   "id": 1
6 }
```



Q1. Draw or describe the request flow for POST /api/rides using controller → service → repository responsibilities.

RideController

↓

RideService

↓

DriverRepository

RideRepository

↓

Database

Q2. Why is driver assignment placed in RideService rather than RideController?

- Driver assignment is business logic. Controllers should only handle HTTP requests and responses. Service classes manage business rules.

Q3. Why is this solution both layered and monolithic?

- It is layered because it uses Controller → Service → Repository layers. It is monolithic because everything runs as a single Spring Boot application and deployment.

Q4. What makes the solution a modular monolith rather than a Big Ball of Mud?

- Driver and Ride are separated into independent modules with clear responsibilities and package boundaries, avoiding tightly coupled code.

Q5. What response does your API return when no driver is available? Explain why it is suitable.

- The API returns: 409 Conflict
with: {
"message": "No driver available"
}
• This clearly informs the client that the request cannot be fulfilled due to resource availability.

Q6. Name one change that would be easier because Driver and Ride are separated into modules

- Adding new driver-related features such as driver ratings would be easier because Driver functionality is already isolated in its own module.